Laboratorio di Calcolo per Fisici

Esercitazione n. 2, 19-20 Ottobre

Canale A-Dec 2026-2027, Docenti: G. Artoni, L. Caprini

Lo scopo della seconda esercitazione di laboratorio è quello di fare pratica con le librerie *matplotlib* e *numpy* per graficare dei dati, e con i primi comandi C.

Fate il login utilizzando `lcgaXXX` sia come *userid* sia come *password*, dove `XXX` è il codice a tre cifre associato al vostro gruppo, così come indicato nella comunicazione dei gruppi.
In quanto segue, si assume che la vostra **`home`** sia `/home/studenti/lcgaXXX`. L'indirizzo potrebbe essere diverso, ma potete controllarlo immediatamente aprendo il terminale ed utilizzando il comando `pwd`.

▶ Prima parte (obbligatoria)

1. Aprite una finestra di *terminale*.
2. Nella vostra home create una cartella chiamata `EX2` e posizionatevi all'interno di quest'ultima. Tutti i file, gli script Python e i codici C dovranno essere contenuti all'interno di questa cartella.

Aprire un editor e creare un file di testo
Per creare uno script Python o un file C conviene aprire l'editor di testo direttamente da terminale. Se usate emacs (un editor di testo antico ma consigliato perché è ciò userete anche l'anno prossimo), questo si può fare dalla riga di comando in questo modo:

```
$ emacs &
```

dove il simbolo `$` indica il `prompt`, mentre il simbolo `&` indica che volete lanciare il programma *in background*, cioè senza bloccare il terminale (provate a non usarlo!). Dopo aver dato il comando si aprirà una finestra di emacs in cui potrete scrivere ciò che volete. Per salvare ciò che scrivete in un file dovrete usare la voce di menu *File* → *Save as...*. **Nota Bene:** la colorazione del vostro codice (*syntax highlighting*) apparirà solo dopo aver dato un nome con la giusta estensione al file (*es.* `prova.py` o `prova.c`).
Per aprire un editor e creare direttamente un file (e avere a disposizione la colorazione fin dall'inizio) passate come parametro il nome del file a emacs:

```
$ emacs prova.py &
```

Qualunque modo scegliate per creare i vostri file, ricordatevi di salvare spesso!

► **Esercizio 1: Lanciare l'interprete python**
Digitate sul vostro terminale il comando

```
python3
```

e premete invio. Noterete che sul terminale compariranno tre freccette `>>>`. Digitate ora il comando python

```
print("Ciao Mondo")
```

e premete invio. Osservate il risultato. Per chiudere l'interprete python digitate il comando

```
exit()
```

Un altro modo è quello di creare uno script in python da far interpretare. Aprite un editor (vedi box all'inizio) e salvate il testo seguente in un file `ex1.py`

```
# Lo script scrive su terminale la frase "Ciao Mondo"
print("Ciao Mondo")
```

Nel terminale scrivete il comando

```
python3 ex1.py
```

e premete invio. Osservate il risultato.

► **Esercizio 2: Il modulo pyplot e numpy**
Questo esercizio mostra come importare il modulo `pyplot` in `matplotlib` per graficare dei dati e il modulo `numpy` per lavorare con dati e funzioni matematiche. Salvate in un file `ex2.py` le seguenti istruzioni:

```
1 # Lo script grafica una parabola
2 import matplotlib.pyplot as plt
3 import numpy as np
4 x = np.linspace(-4, 4, 50, endpoint=True)
5 y = x * x
6 plt.plot(x, y, 'o')
7 plt.show()
8
```

Listato 1: `ex2.py`

Lanciate lo script e osservate il risultato e successivamente fate delle prove modificando il numero dei punti e gli estremi della curva.

► Esercizio 3: lettura dei dati da file

Questo esercizio mostra come importare dei dati contenuti in un file di testo per graficarlo. Copiate ciò che segue in un file `dati.dat`, anche senza incolonnare in maniera ordinata:

```
#x  #y
-4  16
-3   9
-2   4
-1   1
 0   0
 1   1
 2   4
 3   9
 4  16

```

Listato 2: `dati.dat`

Copiate in un file `ex3.py` il seguente script:

```python
# Un primo grafico in python
import matplotlib.pyplot as plt
import numpy as np
x, y = np.loadtxt('dati.dat', unpack=True, usecols=(0,1))
plt.plot(x, y, '-o')
plt.savefig('grafico.png')   # stampa il grafico in grafico.png
plt.show()

```

Listato 3: `ex3.py`

Lanciate lo script e osservate il risultato.

► Esercizio 3-1 (facoltativo): lettura dei dati da file

Aggiungete ora una terza colonna nel file `dati.dat`.

```
#x  #y  #z
-4  16  -64
-3   9  -27
-2   4  -8
-1   1  -1
 0   0   0
 1   1   1
 2   4   8
 3   9  27
 4  16  64

```

Listato 4: `dati.dat` modificato

Modificate lo script `ex3.py` in modo da leggere anche la terza colonna come mostrato di seguito

```
# Un primo grafico in python
import matplotlib.pyplot as plt
import numpy as np
x, y, z = np.loadtxt('dati.dat', unpack=True, usecols=(0,1,2))
plt.plot(x, z, '-o')
plt.savefig('grafico1.png')
plt.show()

```

Listato 5: `ex3.py` modificato

Apportate le seguenti modifiche:

1. Modificate il codice in modo da prendere in input solo la seconda e terza colonna di `dati.dat`.
2. Modificate il codice in modo da leggere tutte e tre le colonne da `dati.dat` e graficate sia y in funzione di x che z in funzione di y. Il secondo grafico si ottiene aggiungendo una seconda istruzione `plt.plot(...)` subito dopo la prima.

► Esercizio 4: personalizzazione dei grafici

Con questo esempio vedremo alcune opzioni per migliorare la leggibilità di un grafico. A partire dallo script creato in precedenza, modificate il comando `plt.plot(...)` come mostrato di seguito:

```
import matplotlib.pyplot as plt
import numpy as np
x, y = np.loadtxt('dati.dat', unpack=True, usecols=(0,1))
plt.plot(x, y, linewidth=4, color='b', linestyle='-', marker='s')
plt.savefig('grafico.png')
plt.show()

```

Listato 6: `ex4.py`

Salvate e nominate il file come `ex4.py`. Lanciate lo script e osservate il risultato. Modificate i parametri della funzione `plt.plot()` usando le opzioni in Fig. 1 (in fondo al testo).

► Esercizio 4-1: modificare l'intervallo del grafico

Modificate lo script precedente (`ex4.py`) aggiungendo la seguente riga di codice prima dell'istruzione per fare il grafico (non c'è bisogno di copiare i commenti)

```
plt.xlim(-3.1, 3.1) # imposta l'intervallo su x tra -3.1 e 3.1
plt.ylim(-0.5, 10.) # imposta l'intervallo su y tra -0.5 e 10.0

```

Listato 7: `ex4.py` modificato

Modificate i campi delle due istruzioni per variare l'intervallo dell'asse x e y.

► Esercizio 5: Etichette degli assi, titolo e legenda

In questo esempio vedremo come aggiungere delle etichette sull'asse x e y, dare un titolo al grafico e graficare la legenda. Copiate lo script `ex4.py` in un nuovo file di testo e modificatelo come mostrato di seguito (non c'è bisogno di copiare i commenti):

```python
import matplotlib.pyplot as plt
import numpy as np

x, y = np.loadtxt('dati.dat', unpack=True, usecols=(0, 1))
plt.title('parabola') # titolo del grafico
plt.xlabel('asse x') # etichetta (label) asse x
plt.ylabel('asse y') # etichetta asse y
plt.plot(x, y, linewidth=2, color='b', linestyle='-', marker='s', label='dati') # il campo label rappresenta la legenda
plt.legend()  # abilita la legenda (va sempre dopo plt.plot())
plt.xlim(-3.1, 3.1)
plt.ylim(-0.5, 10)
plt.savefig('grafico.png')
plt.show()

```

Listato 8: `ex5.py`

Chiamate il file creato `ex5.py`. Lanciate lo script e osservate il risultato. Provate a modificare il titolo, la legenda e i nomi degli assi. *Opzionale:* Graficate assieme ai punti anche la funzione $y = x^2$ come visto nel secondo esercizio (attenzione a non chiamare variabili diverse con lo stesso nome: mi riferisco in particolare alle varie `x` e `y`). Aggiungete la legenda anche per questa seconda curva.

► **Seconda parte (obbligatoria)**

1. Aprite con l'editor di testo un file chiamato `retta.c`.
2. All'interno del file scrivete un codice C che, a partire dalle coordinate di due punti $(x_1, y_1); (x_2, y_2)$ scelti a piacimento, stampi su schermo il coefficiente angolare `m` e il termine noto `q` della retta passante per quei punti.
3. Per controllare che i due parametri siano stati calcolati correttamente, stampate su schermo il valore di $mx_1 + q$ controllando che il risultato dia y_1.
4. Provate a cambiare i valori dei due punti per controllare che i risultati ottenuti siano sempre coerenti.
5. Salvate (a mano!) i due punti usati per calcolare `m` e `q` in un file chiamato `punti.dat` nel formato

```
#x  #y
x1  y1
x2  y2

```

Listato 9: `punti.dat`

6. Create uno script python chiamato `retta.py` che legga il file `punti.dat` e grafichi i punti (i due punti non devono essere uniti da una retta). Corredate il grafico di titolo e nomi degli assi.
7. Graficate assieme ai punti la retta avente coefficiente angolare m e termine noto q calcolata con il codice C. Se necessario modificate l'intervallo sull'asse x e y dove visualizzare i dati.

8. Dopo aver controllato che la retta passi per i due punti, salvate il grafico in un'immagine chiamata `retta.png`.

Suggerimento: Capiamo prima di tutto che cosa richiede l'esercizio. Quello che si vuole calcolare è l'equazione di una retta passante per due punti, quindi prima di scrivere qualsiasi istruzione in C, abbiamo bisogno di carta e penna per ricavare l'espressione per il coefficiente angolare e il termine noto. Per farlo va risolto il sistema

$$\begin{aligned} y_1 &= mx_1 + q \\ y_2 &= mx_2 + q \end{aligned} \tag{1}$$

per m e q. Una volta ottenuti i valori di m e q possiamo pensare a quante variabili abbiamo bisogno per scrivere il nostro codice: quattro per le coordinate dei due punti $(x_1, y_1); (x_2, y_2)$ (di che tipo?), una per il coefficiente angolare e una per il termine noto. Come abbiamo già visto a lezione, queste ultime due variabili sono superflue in quanto potremmo scrivere direttamente l'espressione per il coefficiente angolare e il termine noto nella funzione di stampa, ma come già ripetuto diverse volte il codice deve essere chiaro e leggibile. Dichiariamo quindi variabili intermedie per rendere il codice più semplice da seguire.
Ora non rimane che assegnare un valore alle variabili che contengono le coordinate e scrivere l'espressione per m e q che avete calcolato in precedenza. Dovete poi stampare il risultato usando la funzione di stampa `printf()`, ricordandovi di usare i descrittori associati al tipo di `m` e `q`.
Lo schema del codice dovrà quindi essere:

```c
#include .... //file d'intestazione per l'I/O stdio.h

int main(){
  //corpo del programma
}

```

Listato 10: Schema di `retta.c`

► **Terza parte (facoltativa)**

Si grafichi la funzione $y(x) = (12.0/5.0) \cdot x - 12.5$ con x generato nell'intervallo $[-10.0, 10.0]$ (con l'estremo destro incluso). Il grafico deve contenere una legenda, un titolo e delle etichette sull'asse x e y. Il numero di punti con cui la curva deve essere rappresentata è 22. Il range lungo l'asse x del grafico deve essere $[-6.25, 6.25]$ mentre sull'asse y $[-27.5, 2.6]$. Giocate con gli stili, colori, simboli ecc. e salvate il grafico come un file .png dal nome `retta.png`.

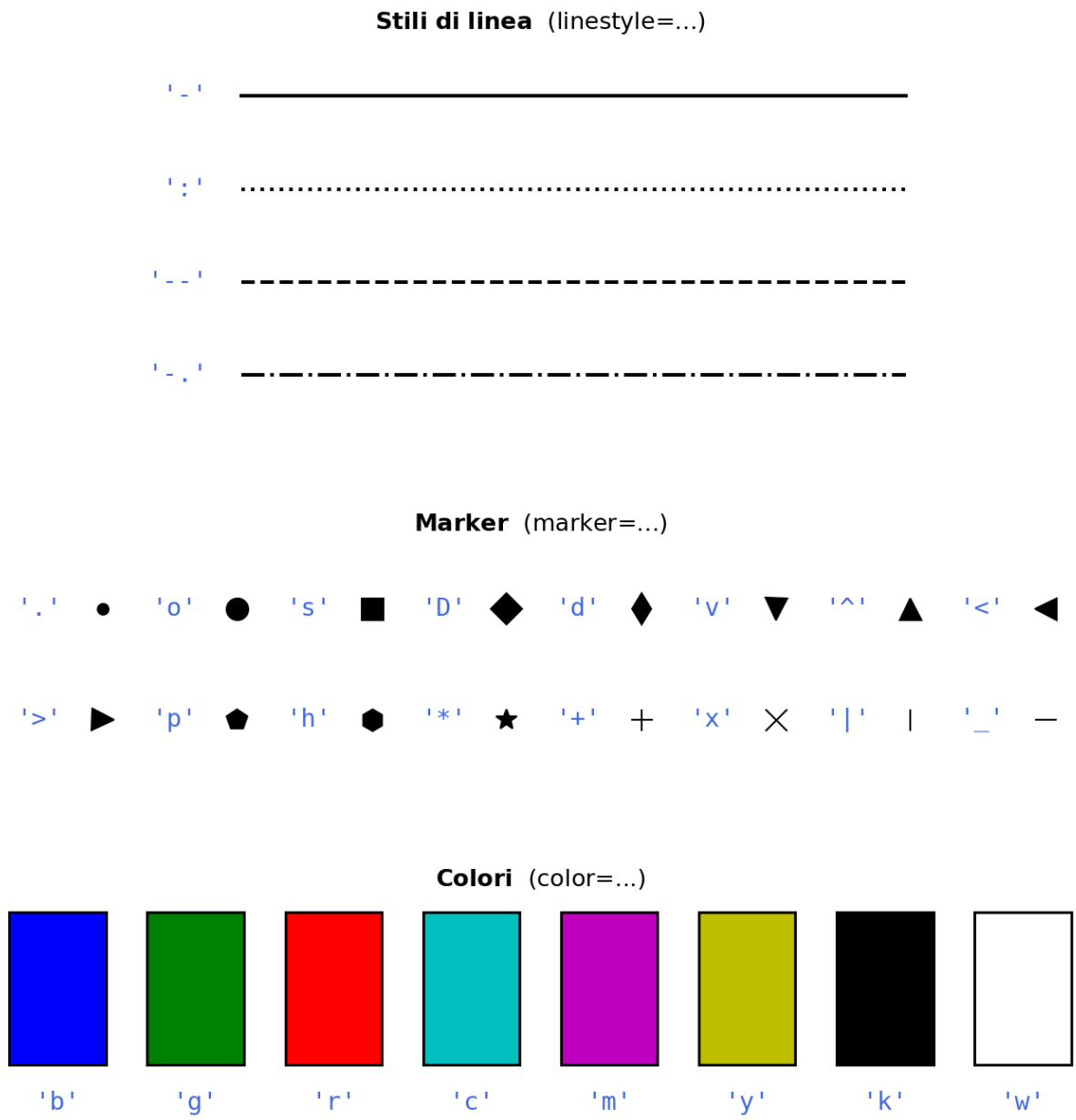


Figura 1: Opzioni per personalizzare lo stile di un grafico